Digital VLSI Design

4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Report

Vidvathama R (2024122002) & Anay Rungta (2024102050)

Technology: SkyWater SKY130, `sky130_fd_sc_hd`, typical corner `tt_025C_1v80`.
Tools: Icarus Verilog 13.0, Yosys 0.69, OpenSTA 2.7.0, LibreLane 3.0.16 (OpenROAD, Magic, Netgen, KLayout).

Summary

RTL simulation (256 vectors)	`PASS: 256 vectors checked, 0 errors`
Gate-level netlist simulation	`PASS: 256 vectors checked, 0 errors`
Cells / area after synthesis	63 cells (16 flip-flops), 740.71 μm^2
$T_{\min}$ / $F_{\max}$ (pre-layout STA)	2.604 ns / 384.0 MHz
LibreLane CLOCK_PERIOD	7.5 ns, 9 run(s)
Post-route $F_{\max}$	141.9 MHz
DRC / LVS errors	0 / 0

1 Part A: RTL design and verification

Design

`rtl/full_adder.v` implements $sum = a \oplus b \oplus c_{\text{in}}$ and $c_{\text{out}} = ab + ac_{\text{in}} + bc_{\text{in}}$ with `assign` statements. `rtl/mult_array.v` registers `a` and `b` into `a_r`, `b_r` (8 flip-flops) and forms the 16 partial products $pp_i[j] = a_r[j] \cdot b_r[i]$. It then builds rows 1–3 from 12 explicitly instantiated `full_adder`s (`fa_r1_0` … `fa_r3_3`), wired exactly as in the table of Section 2. The product is registered into `p` (8 flip-flops). All 16 flip-flops use the asynchronous active-low reset `rst_n`. The design does not use the `*` operator.

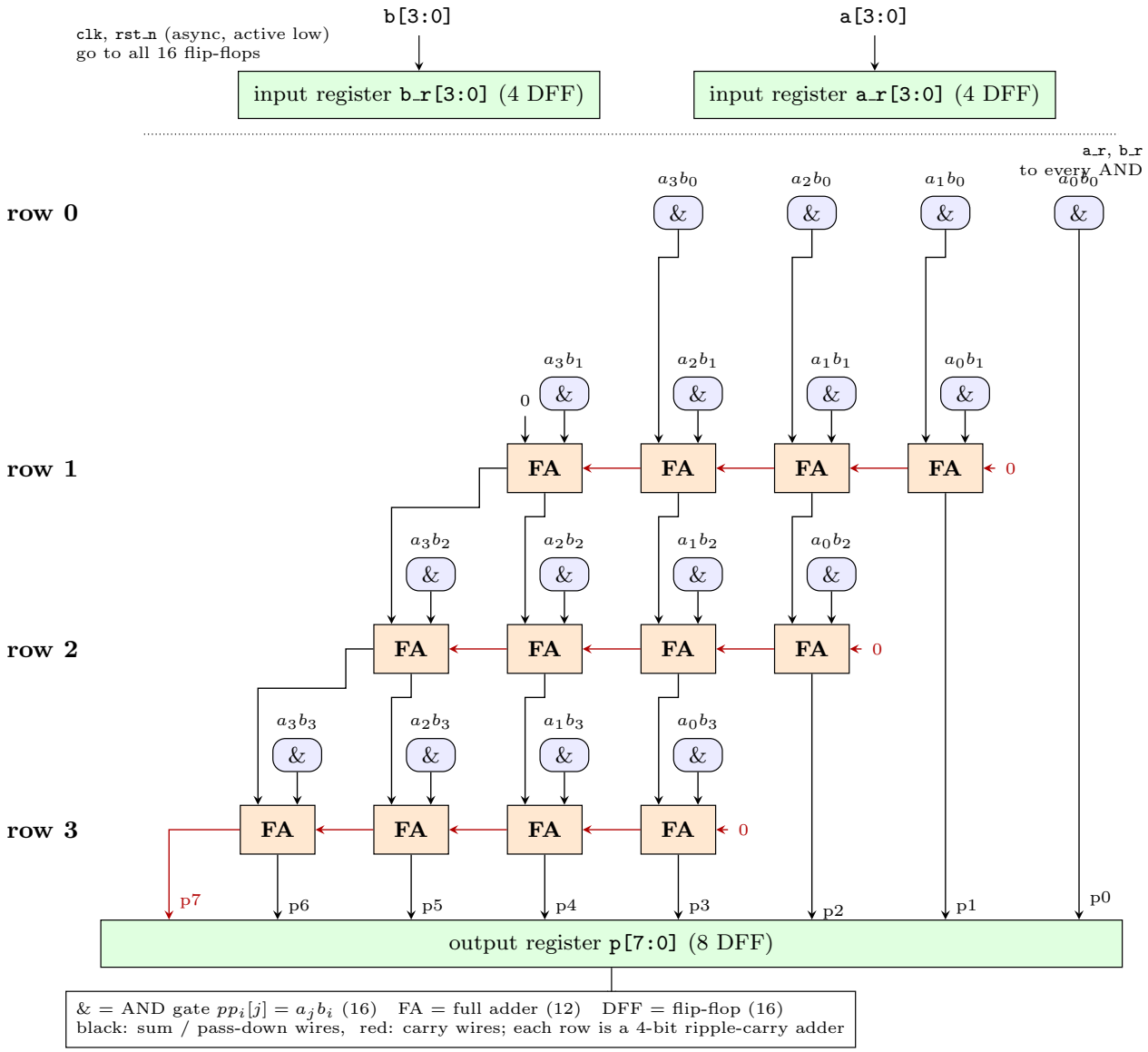


Figure 1: Block diagram: 16 AND gates (&), 12 full adders in three ripple-carry rows, 8 input and 8 output flip-flops. Columns are aligned by bit weight, so each sum passes straight down to the next row and each row's final carry passes down and to the left.

Testbench and results

`tb/tb_mult.v` generates a 10 ns clock and holds `rst_n` low for the first two cycles. It then applies all 256 (a, b) pairs. Each pair is applied on a falling edge, the testbench waits two rising edges (the latency), and on the next falling edge it compares `p` with `a*b`. It prints every mismatch and one summary line, and writes `mult.vcd`.

```
$ iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v && vvp sim
PASS: 256 vectors checked, 0 errors
```

Error-injection test (task 6): with `sum` temporarily changed to `a ^ b`:

```
MISMATCH at 1060 ns: a=3 b=3 p=5 expected=9
MISMATCH at 1120 ns: a=3 b=6 p=10 expected=18
...                                  (95 mismatch lines in total)
FAIL: 256 vectors checked, 95 errors
```

The change was then undone, and the testbench printed `PASS` again. 95 of the 256 products are wrong. These are exactly the cases where some full adder sees a carry-in of 1. Products that never generate

a carry (e.g. any product with $a = 0$, $b = 0$ or a single one-bit partial product) are still correct.

Hand-worked example (task 5)

We used Vidvathama R's roll number, 2024122002. Its last digit is 2, so $A = 2 + 5 = 7$. Because $A = 7$, the rule says to use $B = 9$. So $A = 7 = 0111_2$ and $B = 9 = 1001_2$.

```
        0 1 1 1     A = 7
      x 1 0 0 1     B = 9
  ---------------
        0 1 1 1     pp0 = A AND b0 (b0 = 1)
      0 0 0 0       pp1 = A AND b1 (b1 = 0), shifted 1
    0 0 0 0         pp2 = A AND b2 (b2 = 0), shifted 2
  0 1 1 1           pp3 = A AND b3 (b3 = 1), shifted 3
  ---------------
  0 1 1 1 1 1 1     P = 0011 1111 = 63
```

The same multiplication through the rows of the array (bits written MSB first):

Row	pp_i	input from above	sum s_i, carry $c_i[4]$	product bit
0	0111	–	–	$p[0] = pp_0[0] = 1$, $r_0 = 0011$
1	0000	$r_0 = 0011$	$s_1 = 0011$, $c_1[4] = 0$	$p[1] = 1$, $r_1 = 0001$
2	0000	$r_1 = 0001$	$s_2 = 0001$, $c_2[4] = 0$	$p[2] = 1$, $r_2 = 0000$
3	0111	$r_2 = 0000$	$s_3 = 0111$, $c_3[4] = 0$	$p[3] = 1$, $p[7{:}4] = 0011$

$p = 0011\,1111_2 = 63$. In the waveform (Fig. 2), $a = 7$, $b = 9$ is applied at 2440 ns. Edge 1 (2445 ns) loads it into `a_r`/`b_r`, and edge 2 (2455 ns) loads 63 into `p`. That is a latency of 2 clock cycles.

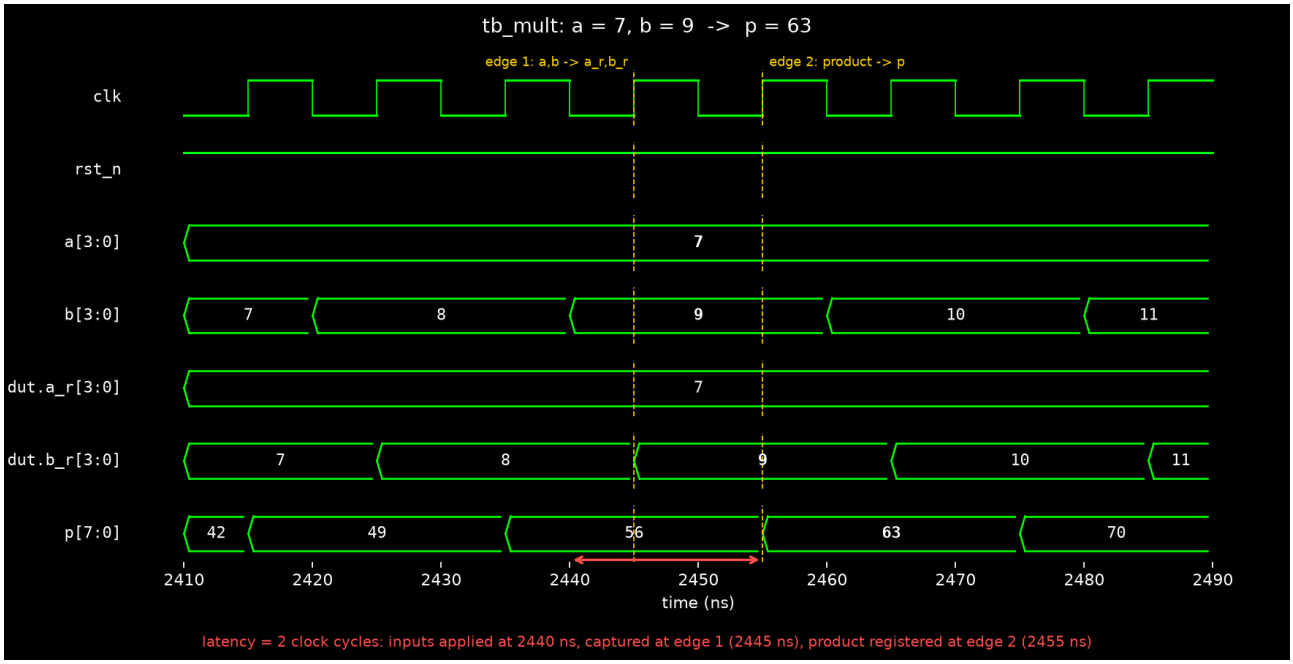


Figure 2: Waveform from `mult.vcd` (RTL simulation) for $a = 7$, $b = 9$, with the two capturing clock edges marked. The internal registers `dut.a_r`/`dut.b_r` are included to show edge 1.

2 Part B: Logic synthesis with Yosys

`synth/synth.tcl` runs the seven required steps: `read_verilog`, `synth -top mult_array -flatten`, `dfflibmap`, `abc`, `opt_clean -purge`, `tee -o ... stat -liberty`, `write_verilog -noattr`.

Metric	Value
Total number of cells	63
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	740.710
Flip-flop area (μm^2)	$16 \times 25.024 = 400.384$
Flip-flop area (% of total)	54.05 %
The 3 most-used cell types	`dfrtp_1` ×16, `nand2_1` ×8, `a21oi_1` ×6
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

The other cells are `xnor2_1`×4, `maj3_1`×3, `nand3_1`×3, `xor2_1`×3, `xnor3_1`×2, `and2_0`×2, `clkinv_1`×2, `lpflow_isobufsrc_1`×2, and one each of `xor3_1`, `a22oi_1`, `and3_1`, `and4_1`, `nand2b_1`, `nand4_1`, `nor2_1`, `nor2b_1`, `nor3_1`, `o2111ai_1`, `o21ai_0` and `o311ai_0`. The gate-level simulation used `-DFUNCTIONAL -DUNIT_DELAY=#1` with the SKY130 `primitives.v` and `sky130_fd_sc_hd.v` models. It reproduced the RTL result exactly, so synthesis did not change the behaviour.

3 Part C: Static timing analysis with OpenSTA

`sta/sta.tcl` performs the seven required steps (typical corner, ideal clock, 1.0 ns input and output delays). The period is taken from `CLK_PERIOD`.

Metric	Value
Worst slack at $T = 10$ ns	+7.396 ns (`report_worst_slack`: 7.40)
$T_{\min} = T - \text{WNS}$	$10 - 7.396 = 2.604$ ns
$F_{\max} = 1000/T_{\min}$	384.0 MHz
Slack at $T = T_{\min}$	+0.006 ns at $T = 2.61$ ns (−0.004 ns at 2.60 ns)
Critical path start point	`a_r[1]` (cell `_095_`, `dfrtp_1`, clock pin)
Critical path end point	`p[6]` (cell `_092_`, `dfrtp_1`, D pin) = sum of `fa_r3_3`
Number of cells on the critical path	8 combinational cells (10 including the two flip-flops)
Latency $= 2 \times T_{\min}$	5.21 ns
Throughput	384 million results per second

$T_{\min} = 2.604$ ns rounds to 2.60 ns. At that period the slack is −0.004 ns, which is essentially zero but on the failing side. We therefore rounded up to 2.61 ns (+0.006 ns, met), and that run produced the saved `sta/mult_array_timing.rpt`.

Cells on the critical path (in order)

#	Instance	Cell	Function in the multiplier	Delay (ns)	Arrival (ns)
–	`_095_`	`dfrtp_1`	launch FF `a_r[1]` (clk→Q)	0.398	0.398
1	`_039_`	`clkinv_1`	invert `a_r[1]` (partial-product logic)	0.074	0.472
2	`_047_`	`o311ai_0`	partial products + row-1 adder logic merged by ABC	0.348	0.820
3	`_050_`	`a21oi_1`	low-order carry (rows 1–2)	0.172	0.992
4	`_063_`	`maj3_1`	full-adder carry	0.381	1.373
5	`_064_`	`xnor2_1`	full-adder sum	0.160	1.534
6	`_066_`	`maj3_1`	full-adder carry	0.385	1.918
7	`_075_`	`maj3_1`	full-adder carry	0.377	2.295
8	`_079_`	`xnor3_1`	full-adder sum $s_3[3]$ (`fa_r3_3`)	0.175	2.471
–	`_092_`	`dfrtp_1`	capture FF `p[6]`, setup time	0.134	(2.604)

Full-adder cells (task 3): 5 of the 8 cells are full-adder cells: three `maj3` carry cells and the `xnor2`/`xnor3` sum cells. The first three cells (`clkinv`, `o311ai`, `a21oi`) implement AND gates of the partial products, which ABC merged with the first full adders of rows 1 and 2. The section 2 estimate is about $2N = 8$ full-adder delays. The real path crosses roughly 5–6 full-adder stages, plus some

merged AND logic. It is shorter than the estimate because flattening removes logic: $c_i[0] = 0$ and $r_0[3] = 0$ turn several full adders into half adders, and AND gates are absorbed into complex gates. Coincidentally the cell count (8) matches $2N$, but most cells implement only half a full adder (the sum or the carry). The combinational delay is 2.07 ns, about 0.26 ns per cell.

4 Part D: Physical design with LibreLane

`pnr/config.json` sets `DESIGN_NAME = "mult_array"`, the two RTL files, `CLOCK_PORT = "clk"`, `FP_SIZING = "absolute"` and `DIE_AREA = [0, 0, 150, 150]`. It also sets `CLOCK_PERIOD` $= 1.2 \times T_{\min} = 1.2 \times 2.604 = 3.125$ ns, rounded up to the next 0.5 ns, which is **3.5 ns**. The flow was run with `librelane --manual-pdk --pdk-root $PDK_ROOT config.json` (LibreLane 3.0.16, PDK revision `8afc8346`).

Metric	Value
CLOCK_PERIOD used (ns)	7.5
Number of runs needed	9
Core area (μm^2)	17 759.5 (die area 22 500 μm^2)
Cells after synthesis (from Part B)	63
Cells after place and route	4831 (339 without fill/decap cells)
Kinds of cells added	tap, decap (incl. endcaps), fill, clock and I/O buffers (details below)
Wirelength (μm)	2516
Setup slack (ns)	+0.454 (worst of 9 corners, `max_ss_100C_1v60`); +3.817 at `nom_tt`
Post-route $F_{\max}$ (MHz)	141.9
$F_{\max}$ from Part C (MHz)	384.0
DRC errors	0
LVS errors	0

Run history (task 1). LibreLane's `timing__setup__ws` is the worst setup slack over all nine signoff corners ({min, nom, max} parasitics × {tt_025C_1v80, ss_100C_1v60, ff_n40C_1v95}). Every run had 0 DRC and 0 LVS errors. Setup timing failed in the slow corner until 7.5 ns, so we increased CLOCK_PERIOD by 0.5 ns each time:

CLOCK_PERIOD (ns)	`setup__ws` (ns)	tt setup WS (ns)	Instances	Wirelength (μm)	Result
3.5	−2.093	+0.493	4791	2602	fail (ss)
4.0	−1.593	+0.993	4791	2602	fail (ss)
4.5	−1.093	+1.493	4791	2601	fail (ss)
5.0	−0.596	+1.991	4794	2612	fail (ss)
5.5	−0.104	+2.487	4797	2584	fail (ss)
6.0	−0.043	+2.795	4823	2530	fail (ss)
6.5	−0.155	+3.050	4830	2508	fail (ss)
7.0	−0.046	+3.317	4831	2516	fail (ss)
7.5	**+0.454**	+3.817	4831	2516	**pass**

The design already met timing at 3.5 ns in the typical corner (+0.49 ns). Only the slow-slow corner (1.60 V, 100 °C) needed the longer period. The slack does not improve by exactly 0.5 ns per step between 5.5 and 7.0 ns. LibreLane's own synthesis is timing-driven by CLOCK_PERIOD (`SYNTH_STRATEGY = "AREA 0"`), so with a looser clock ABC picks smaller, slower gates and a slightly different netlist (hence the changing instance count). The design also has 11 max-slew warnings in the ss corner, all on the `rst_n` fan-out net (0.852 ns against the 0.75 ns limit). They do not affect setup or hold checks or DRC/LVS. Hold slack is +0.120 ns (met).

Post-route $F_{\max}$ (task 3) $= 1000/(\text{CLOCK_PERIOD} - \text{setup slack}) = 1000/(7.5 - 0.454) = 1000/7.046 =$ **141.9** MHz, versus 384.0 MHz in Part C, which is 2.7× lower. Using the typical-corner slack for a like-for-like comparison: $1000/(7.5 - 3.817) = 271.5$ MHz, still 29 % below Part C (see question 4).

Cells added by the flow (task 4). LibreLane re-synthesises the RTL itself and got 71 cells (55 logic + 16 `dfrtp_2`), compared with 63 in Part B. After place and route there are 4831 instances:

Cell	Count	Added by
`sky130_fd_sc_hd__decap_3`	4213	decap / endcap insertion (94 are row endcaps) and fill step
`sky130_fd_sc_hd__fill_1`, `fill_2`	226 + 53	fill insertion (gaps narrower than a decap)
`sky130_fd_sc_hd__tapvpwrvgnd_1`	245	tap-cell insertion
`sky130_fd_sc_hd__clkbuf_16`	3	clock-tree synthesis (1 root + 2 leaf buffers)
`sky130_fd_sc_hd__clkdlybuf4s25_1`	20	resizer: 9 input + 8 output port buffers, 3 fan-out buffers
logic + flip-flops (from synthesis)	71	
Total	4831	

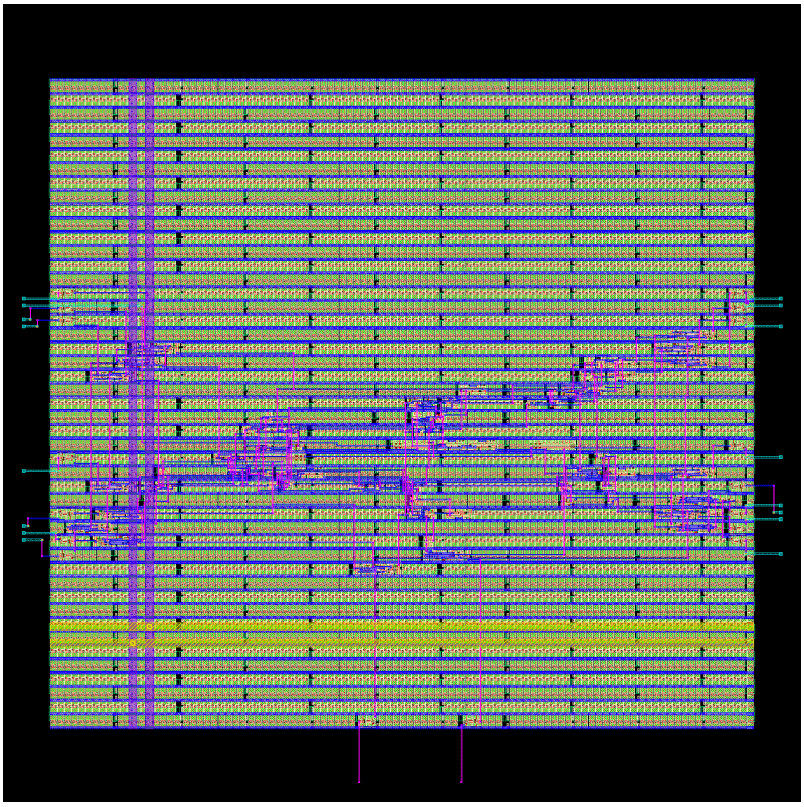


Figure 3: Final layout `mult_array.gds` in KLayout: the whole $150 \times 150\,\mu$m die with the power grid, standard-cell rows and I/O pins.

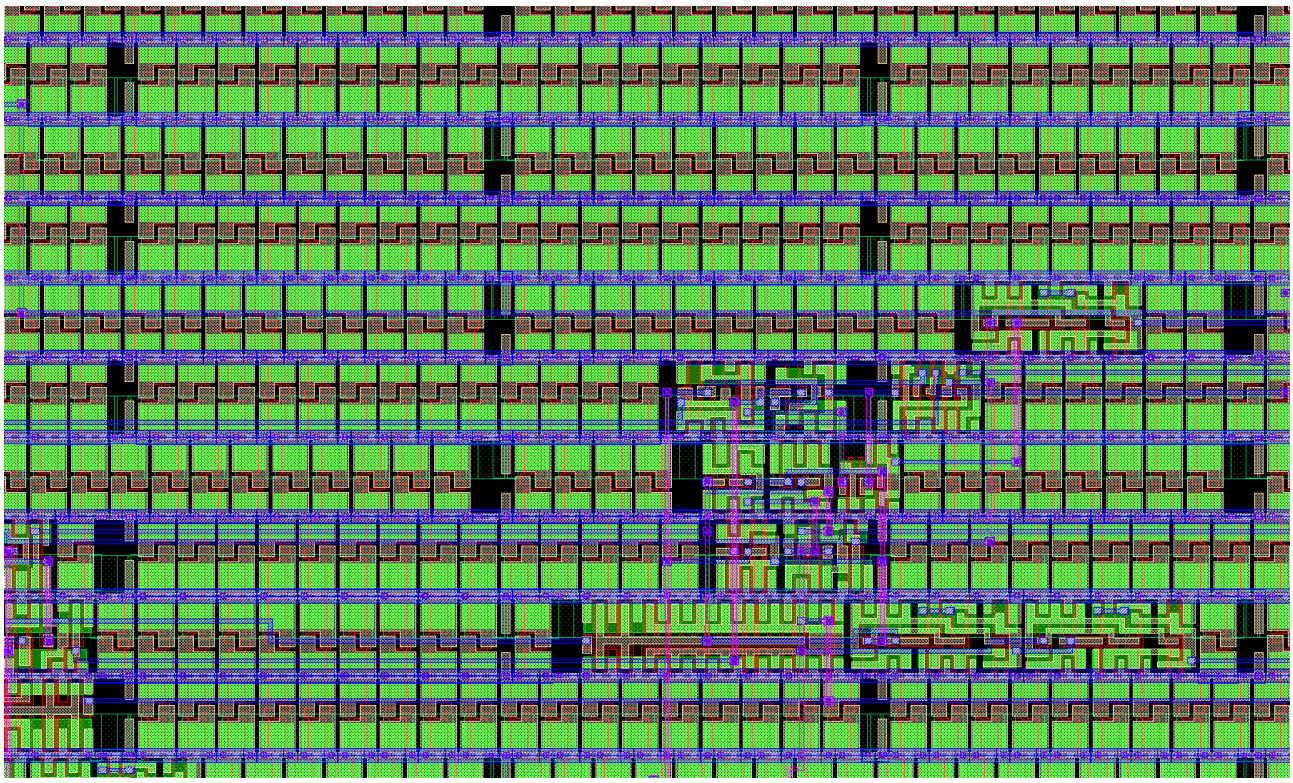


Figure 4: Zoomed view of the same layout: standard-cell rows (with tap, decap and fill cells between the logic cells) and the routed metal wires.

5 Answers to the questions

Q1. Logic cells versus 16 AND + 12 FA. Synthesis produced 63 cells, 16 of them flip-flops, so **47 cells** implement the logic, not $16 + 12 = 28$. There are three reasons the count is not simply $16 + 12$. (i) There is no one-to-one mapping. After `synth -flatten` the full adders are dissolved into a single Boolean network, and ABC re-maps that network to whatever library cells it finds cheapest. A full adder is not one cell here: `abc` does not use the multi-output `sky130_fd_sc_hd__fa_1` cell (ABC warns about this), so each sum becomes an `xor3`/`xnor3` or two `xnor2`s, and each carry becomes a `maj3` or an AOI/OAI combination. (ii) Optimisation removes and merges logic. The constants $c_i[0] = 0$ and $r_0[3] = 0$ turn four full adders into half adders, and many partial-product AND gates are absorbed into complex gates such as `a21oi`, `a22oi`, `o311ai` and `o2111ai`, or into `nand` gates feeding the adders. (iii) Cells of the opposite polarity need extra inverters or helper gates (two `clkinv`, two `lpflow_isobufsrc` used as AND-with-inverted-input). So the count reflects the library's cell mix, not the RTL structure.

Q2. Critical path. The critical path starts at the input register `a_r[1]` (clock pin of flip-flop `_095_`) and ends at the output register `p[6]` (D pin of `_092_`), which captures the sum $s_3[3]$ of the last full adder `fa_r3_3`. It contains **8 combinational cells** (`clkinv`, `o311ai`, `a21oi`, `maj3`, `xnor2`, `maj3`, `maj3`, `xnor3`) between the two flip-flops, with 2.07 ns of logic delay ($T_{\min} = 2.604$ ns including clk→Q 0.398 ns and setup 0.134 ns). The $2N = 8$ full-adder estimate counts complete full adders along the carry ripple and down the rows. Our path has 8 *cells*, but each cell is only about half a full adder (a carry or a sum function), and the first three cells are partial-product logic merged with the first adders. In full-adder terms it is therefore about 5–6 stages, shorter than $2N$. The estimate is an upper-bound model of the unoptimised array: synthesis removes the half-adder positions ($c_i[0] = 0$, $r_0[3] = 0$) and uses fast complex cells (`maj3`, `xnor3`). The delay still follows the predicted shape: it runs diagonally through the carry chains of rows 1–3 to the most significant sum bits.

Q3. Flip-flop share of area. The flip-flops take 400.38/740.71 = **54.05** % of the cell area: 16 `dfrtp_1` cells at 25.02 μm^2 each. One `dfrtp_1` has about the area of 6–7 `nand2_1` cells (3.75 μm^2). It contains a master and a slave latch, clock inverters and the asynchronous-reset transistors, about 25 transistors in total. The combinational part of a 4-bit multiplier is tiny: 47 small gates, 340 μm^2. The number of registers grows linearly with width ($4N = 16$) while the array logic grows quadratically (N^2 ANDs and $N(N-1)$ FAs). For small N the registers therefore dominate. For an 8-bit design (32 flip-flops versus 64 ANDs and 56 FAs) the percentage would drop to roughly a third.

Q4. Why the post-route $F_{\max}$ is lower, and which cells were added. The post-route $F_{\max}$ is 141.9 MHz (worst corner) or 271.5 MHz (typical corner), against 384.0 MHz in Part C. Part C was an idealised analysis: typical corner only, ideal clock (zero latency and skew), no wires, no uncertainty, and 1 ns I/O delays. Signoff STA after routing adds the following. (a) *Extracted wire RC*: every net now has real resistance and capacitance from the routed metal (SPEF), which slows the gates and degrades slews. (b) A *real clock tree*: 3 `clkbuf_16` add 0.33 ns (tt) to 0.59 ns (ss) of clock insertion delay. Most of it cancels between the launch and capture flip-flops; what remains is clock skew between them. (c) *Margins*: 0.25 ns clock uncertainty, a 5 % timing derate, I/O delays of 20 % of the period, and a 33 fF output load. (d) *Multi-corner signoff*: the reported `setup__ws` comes from the ss corner (1.60 V, 100 °C), where logic is about 2× slower than at tt (the same critical path takes 3.30 ns at tt and 6.46 ns at ss). This is the main reason for the 2.7× drop. (e) LibreLane re-synthesises with an area-oriented strategy, which gives a slower netlist than our Part B one.

The flow added four kinds of cells. **Tap cells** (245 `tapvpwrvgnd_1`) tie the n-well to VPWR and the p-substrate to VGND at regular spacing; this prevents latch-up of the parasitic thyristor in CMOS. **Decap cells** (4213 `decap_3`, including 94 endcaps at the row ends) are on-chip capacitors between VPWR and VGND. They supply charge locally when many gates switch, which reduces supply noise and IR drop. Endcaps also terminate each row cleanly for DRC. **Fill cells** (279 `fill_1/fill_2`) fill the remaining gaps so that the n-well, implant layers and power rails are continuous along every row, which the DRC rules and manufacturing density require. **Buffers**: 3 `clkbuf_16` build the clock tree so all 16 flip-flops receive the clock at nearly the same time. The resizer added 20 `clkdlybuf4s25_1`: 17 isolate the I/O ports from the internal logic and drive the output load, and 3 repair fan-out on high-fanout nets such as `rst_n`.

Q5. DRC and LVS. *DRC* (design rule check, here run by Magic and KLayout) checks the layout geometry against the foundry's manufacturing rules: minimum width and spacing of every layer, enclosure of vias and contacts, overlaps, well spacing, antenna ratios and density. These rules encode what the lithography and etch process can reliably reproduce. *LVS* (layout versus schematic, here Netgen) extracts the transistors and their connectivity from the layout (with Magic) and compares that circuit with the gate-level netlist. It checks that every device, net and pin matches, with no opens, shorts or missing or extra connections. Both must pass before tape-out because they catch different failures. A layout can be DRC-clean yet wired wrongly, which is an LVS error, and the chip would then compute the wrong function. A layout can also be logically correct yet violate DRC, so the shapes print with shorts, opens or reliability problems and yield drops or the chip fails. Masks cost a great deal and cannot be fixed afterwards, so foundries accept only DRC-clean, LVS-clean GDS. Our final layout has 0 DRC errors (Magic 0, KLayout 0) and 0 LVS errors.

Q6. Scaling to 8 bits. For $N = 8$, using the Section 2 formulas: AND gates $N^2 = 64$ (4× the 16 of $N = 4$); full adders $N(N-1) = 8 \cdot 7 = \mathbf{56}$ (versus $4 \cdot 3 = 12$, so $56/12 \approx 4.7\times$ more, quadratic growth); flip-flops $4N = 32$ (2×). Critical path $\approx 2N$ full-adder delays = **16** (versus 8), so the delay **doubles**, linear growth. Using our own STA numbers: the logic delay of the 4-bit design is $T_{\min} - t_{clk\to Q} - t_{\text{setup}} = 2.604 - 0.398 - 0.134 = 2.072$ ns for the "$2N = 8$" path, about 0.259 ns per unit. For $N = 8$ that gives $16 \times 0.259 = 4.14$ ns of logic, so $T_{\min} \approx 4.14 + 0.53 = 4.68$ ns and $F_{\max} \approx 214$ MHz, compared with 384 MHz for 4 bits. The adder area grows about 4.7× while the

speed only halves, which is why wide multipliers use faster structures (Wallace or Dadda trees with a fast final adder) to obtain $O(\log N)$ depth.